Índices de MongoDB 8.0

Antes de estudiar índices: los datos y las consultas que vamos a optimizar

0.1. Por qué hay que empezar por los documentos

Un principiante puede copiar `createIndex` y conseguir que MongoDB responda «OK» sin haber aprendido nada. Para aprender índices primero necesitamos comprender qué estamos buscando, cómo expresamos esa búsqueda y qué significa un resultado correcto.

MongoDB almacena **documentos**. Cada documento tiene campos y valores. Los documentos se reúnen en una **colección**, y las colecciones pertenecen a una **base de datos**. En este curso utilizaremos la base de datos `curso_indices`, cuya colección principal se llama `ventas`.

Ejemplo de un documento:

```
{
  numero: 101,
  clienteId: 809,
  estado: "pagada",
  fecha: ISODate("2025-01-12T00:00:00Z"),
  importe: 72.50,
  etiquetas: ["web", "urgente"]
}
```

Antes de pensar en índices debemos reconocer que estos campos no son del mismo tipo:

- `numero` y `clienteId` son valores numéricos.
- `estado` es texto; las comillas indican una cadena.
- `fecha` es una fecha representada en el shell mediante `ISODate`.
- `importe` es un número decimal.
- `etiquetas` es un array, es decir, una lista de valores.

MongoDB utiliza un formato binario llamado **BSON** que permite almacenar tipos apropiados para documentos. La sintaxis que vemos en `mongosh` se parece a JavaScript, pero no debemos confundir una fecha con la cadena de texto que contiene sus caracteres.

0.2. Una base de datos no es una pantalla de la aplicación

Una tienda puede mostrar una tabla con diez ventas, pero la colección de MongoDB puede contener 200.000 documentos. La pantalla está mostrando una pequeña selección de los datos.

Piensa en tres capas:

1. **Datos almacenados:** las ventas existentes.
2. **Consulta:** la petición concreta de una aplicación o de una persona.
3. **Resultado:** los documentos que se entregan al cliente.

Los índices influyen en la manera de obtener el resultado, **no cambian por sí solos lo que la aplicación ha pedido**.

0.3. Cómo seleccionar la colección correcta

En `mongosh` podemos establecer una referencia a la colección deseada:

```
const ventas = db.getSiblingDB("curso_indices").ventas;
```

A partir de ahí `ventas.find(...)` y `ventas.countDocuments(...)` actuarán sobre esa colección. El nombre es importante. Si consultas una base distinta, podrías obtener cero documentos sin que exista ningún error de indexación.

0.4. Buscar por igualdad

```
ventas.find({ estado: "pagada" })
```

La orden se lee así: «de la colección de ventas, devuelve los documentos cuyo campo `estado` contiene exactamente el valor `pagada`».

Es distinto de:

```
ventas.find({ estado: "pendiente" })
```

Las dos consultas tienen la misma estructura, pero responden preguntas diferentes.

Pregunta de comprensión: si una venta está pagada, ¿debería aparecer en una consulta por pendientes? Si respondes que sí porque contiene el campo `estado`, aún no has entendido el significado del filtro.

0.5. Comparar valores: mayor, menor e igualdad

Imagina cuatro ventas:

- A: pagada, importe 720.
- B: pagada, importe 200.
- C: pendiente, importe 880.
- D: cancelada, importe 80.

La consulta:

```
ventas.find({ importe: { $gt: 500 } })
```

selecciona ventas con importe superior a 500: A y C.

La consulta:

```
ventas.find({
  estado:"pagada",
  importe:{$gt:500}
})
```

añade una segunda condición: solo A cumple simultáneamente el estado y el importe. Esto enseña algo previo a cualquier índice: **la lógica del filtro determina qué significa un resultado correcto**.

Operadores que utilizarás:

- `$gt`: mayor que.
- `$gte`: mayor o igual.
- `$lt`: menor que.
- `$lte`: menor o igual.

La diferencia entre `$gt:500` y `$gte:500` se nota exactamente en el valor 500. Un índice no corrige una desigualdad mal planteada.

0.6. Ordenar no es filtrar

```
ventas.find({estado:"pagada"}).sort({fecha:-1})
```

La condición `estado` determina qué documentos entran en el conjunto. `sort` establece en qué orden queremos verlos. El valor `-1` pide fecha descendente, es decir, los más recientes primero.

Si solo necesitamos diez:

```
ventas.find({estado:"pagada"})
  .sort({fecha:-1})
  .limit(10)
```

No confundas `limit(10)` con «el servidor solo examina diez documentos». Sin un acceso adecuado puede necesitar recorrer muchísimos candidatos antes de entregar diez.

0.7. Elegir qué campos recibir: la proyección

```
ventas.find(
  {estado:"pagada"},
  {_id:0,numero:1,importe:1}
)
```

El primer objeto sigue siendo el filtro. El segundo indica la **proyección**: queremos ver `numero` e `importe`, pero no `_id`. Esa decisión importa cuando estudiemos consultas cubiertas, porque el índice debe contener lo necesario para responder sin buscar otros campos en cada documento.

0.8. Leer frente a escribir

Estas instrucciones hacen cosas radicalmente distintas:

```
ventas.find({numero:101})
ventas.insertOne({numero:101,estado:"pagada"})
ventas.updateOne({numero:101},{$set:{estado:"enviada"}})
ventas.deleteOne({numero:101})
```

La primera consulta; la segunda crea un documento; la tercera modifica uno; la cuarta intenta borrarlo. En las prácticas no se ejecutan comandos de modificación como entretenimiento: deben tener una finalidad concreta y utilizar datos ficticios.

Al crear índices únicos, por ejemplo, una inserción duplicada puede ser rechazada. Ese fallo podría demostrar que la integridad está protegida.

0.9. Tres preguntas obligatorias antes de utilizar explain

1. ¿Cuál es **la pregunta de negocio** que resuelve la consulta?
2. ¿El resultado es correcto: contiene los documentos esperados?
3. ¿Qué trabajo realizó MongoDB para obtenerlo?

La tercera cuestión es donde entra `explain`. Si las dos primeras no están claras, una tabla de `totalDocsExamined` no ayudará a aprender.

Comprobación inicial

¿Por qué no son equivalentes «devuelve diez ventas» y «examina diez documentos»? ¿Por qué `sort` no implica la creación automática de un índice? ¿Cómo influye una proyección en lo que el cliente recibe?

Si puedes responder estas tres preguntas, estás preparado para comenzar la primera unidad sobre índices.

Unidad 1 — Encontrar una venta entre 200.000

El problema, antes que el comando

Imagina que una empresa guarda 200.000 ventas. Un cliente solicita información sobre la venta número 199.998. El operador necesita localizarla, pero su base de datos no tiene un camino de búsqueda organizado por número. Como en una pila de carpetas desordenadas, el sistema puede verse obligado a examinar documentos sucesivamente. El cliente recibirá una respuesta correcta, pero el esfuerzo puede ser enorme.

Un **índice** es una estructura adicional que MongoDB mantiene ordenada por el valor de uno o varios campos y que permite localizar documentos sin examinar necesariamente toda la colección. No sustituye la colección ni almacena una segunda copia íntegra de cada venta.

Primer documento de ejemplo

```
{
  numero: 199998,
  clienteId: 809,
  estado: "pagada",
  importe: 84.50,
  fecha: ISODate("2025-07-01T00:00:00Z")
}
```

Un documento es una unidad de información. La colección `ventas` agrupa documentos similares. El campo `numero` identifica una venta, mientras `clienteId` identifica a su cliente. Los nombres de campos se escriben exactamente: `numero` no es lo mismo que `número`.

Buscar no significa examinar solo un documento

```
db.ventas.find({ numero: 199998 })
```

El filtro pide las ventas cuyo número sea exactamente 199.998. Aunque reciba un solo resultado, MongoDB puede haber examinado miles de documentos para encontrarlo. **Número de resultados** y **cantidad de trabajo del motor** son conceptos distintos.

Sin un índice útil, el plan puede incluir `COLLSCAN`: recorrer documentos de la colección. Eso no es siempre incorrecto; para una colección diminuta puede ser razonable. Para una búsqueda frecuente en 200.000 registros, conviene comprobar el coste.

Crear un índice

```
db.ventas.createIndex({ numero: 1 })
```

El valor `1` indica orden ascendente de la clave. El `-1` indica descendente. Al crear un índice sobre `numero`, MongoDB dispone de otro camino para localizar una venta. La misma consulta puede pasar a utilizar `IXSCAN`, un recorrido del índice. Si además necesita el documento completo, el plan puede incluir `FETCH`.

Los índices tienen coste: ocupan espacio y añaden trabajo a inserciones, actualizaciones y borrados. Por eso no tiene sentido crear índices para todos los campos sin conocer las consultas habituales de la aplicación.

Cómo saber si sirvió: explain

```
db.ventas.find({ numero: 199998 }).explain("executionStats")
```

Esta orden no crea índices. Nos muestra cómo se resuelve la consulta y qué trabajo ha realizado.

Campo	Qué significa
`nReturned`	Cuántos documentos devolvió la búsqueda
`totalDocsExamined`	Cuántos documentos examinó el motor
`totalKeysExamined`	Cuántas claves del índice examinó
`executionTimeMillis`	Tiempo observado en esa ejecución, no una promesa de tiempo futuro

Una consulta sin índice podría examinar casi 200.000 documentos y devolver uno. Con un índice apropiado podría examinar una sola clave y un documento. **No memorices cifras de tiempo:** la CPU, la caché y la carga cambian cada ejecución. Compara primero resultados y contadores de trabajo.

Qué es una comparación válida

Para comprobar una mejora hay que repetir el mismo filtro, sobre los mismos datos, con la misma proyección y límite. Si antes consultas por número y después por cliente, no puedes atribuir la diferencia al índice.

El laboratorio de la unidad siguiente permite reproducir esta comparación: carga ventas ficticias, observa `COLLSCAN`, crea el índice y repite la búsqueda con `explain`.

Comprueba que lo has comprendido

1. ¿Por qué una consulta que devuelve un registro puede tener un coste elevado?
2. ¿Cuál es la diferencia entre `COLLSCAN` e `IXSCAN`?
3. ¿Qué añade `FETCH` al recorrido de índice?
4. ¿Por qué no crearías diez índices solo porque puedas?
5. Si el número de documentos examinados baja de 200.000 a 1, ¿significa que siempre habrá la misma proporción de mejora en milisegundos?

Respuesta razonada a la primera pregunta: porque encontrar un resultado puede requerir comprobar muchos candidatos. La consulta puede ser correcta y al mismo tiempo innecesariamente costosa.

Siguientes unidades

Se desarrollarán de forma independiente los índices compuestos, orden de campos, regla ESR, multikey, búsqueda de texto, geoespaciales, hashed, unique, TTL y mantenimiento. Cada concepto se presentará con un problema reconocible, un ejemplo mínimo, una ejecución guiada y un criterio para decidir cuándo usarlo.

Referencia: https://www.mongodb.com/docs/v8.0/reference/explain-results/

Unidad 2 — Índices compuestos: cuando la pregunta tiene varias condiciones

1. La situación: no buscamos una venta, buscamos una lista útil

En la unidad anterior buscábamos una venta por su número. Ahora la responsable de una tienda nos pide algo diferente:

> «Enséñame las diez ventas pagadas más recientes cuyo importe sea superior a 500 euros».

En una aplicación de verdad, este tipo de consulta es habitual: combina un estado, un umbral económico, una ordenación y un límite. Es más exigente que recuperar una venta por un identificador exacto, porque MongoDB debe resolver **qué documentos interesan** y **en qué orden entregarlos**.

Una venta sintética del laboratorio contiene, entre otros, estos campos:

```
{
  numero: 812,
  estado: "pagada",
  fecha: ISODate("2025-03-23T00:00:00Z"),
  importe: 812.95
}
```

No es importante memorizar ese ejemplo. Lo importante es identificar qué necesitamos pedir: estado igual a «pagada», importe mayor de 500, orden descendente por fecha y máximo diez resultados.

Esta consulta se expresa así:

```
db.ventas
  .find({ estado: "pagada", importe: { $gt: 500 } })
  .sort({ fecha: -1 })
  .limit(10)
```

Léela paso a paso antes de continuar:

1. `find` selecciona documentos.
2. `estado: "pagada"` exige igualdad: el estado debe coincidir exactamente.
3. `importe: { $gt: 500 }` exige **rango**, no igualdad; `$gt` significa *greater than*, «mayor que».
4. `sort({ fecha: -1 })` quiere los más recientes primero; `-1` significa orden descendente.
5. `limit(10)` permite entregar como máximo diez documentos.

El resultado son diez ventas (si existen al menos diez), pero esa cifra no nos dice cuánto trabajo realizó MongoDB para encontrarlas y ordenarlas.

2. Qué es un índice compuesto

Un **índice compuesto** utiliza varios campos, en un orden concreto. Por ejemplo:

```
db.ventas.createIndex({ estado: 1, fecha: -1 })
```

Este índice no es equivalente a tener un índice independiente en `estado` y otro en `fecha`. Es una **única estructura** cuyas claves están organizadas primero por estado y, dentro de cada valor de estado, por fecha descendente.

Piensa en un archivador de tarjetas:

- Primero se agrupan las tarjetas por estado: canceladas, enviadas, pagadas, pendientes.
- Dentro de cada grupo se ordenan de la fecha más reciente a la más antigua.
- Si pides «pagadas» y las últimas fechas, puedes dirigirte al grupo apropiado y recorrerlo ya ordenado.

Una representación conceptual sería:

```
ESTADO       FECHA
cancelada    2025-12-30
cancelada    2025-12-28
...
pagada       2025-12-31
pagada       2025-12-30
pagada       2025-12-29
...
pendiente    2025-12-31
```

Este esquema es una **explicación conceptual**, no un volcado de las páginas internas del índice MongoDB.

Por qué importa el orden de los campos

Compara estas dos definiciones:

```
{ estado: 1, fecha: -1 }
{ fecha: -1, estado: 1 }
```

Tienen los mismos campos, pero no organizan la información del mismo modo. La primera agrupa por estado y después ordena fechas; la segunda agrupa primero por fecha, independientemente del estado. Eso cambia las consultas que aprovechan mejor el índice.

Conclusión que conviene recordar: en un índice compuesto importa **qué campos elegimos, en qué orden se colocan y para qué consultas se utilizarán**.

3. El prefijo izquierdo: por qué no vale cualquier combinación

Un índice `{ estado: 1, fecha: -1, importe: 1 }` posee varios prefijos útiles:

- `{ estado: 1 }`
- `{ estado: 1, fecha: -1 }`

- La clave completa `{ estado: 1, fecha: -1, importe: 1 }`

¿Por qué llamamos a esto «prefijos»? Porque todas esas combinaciones **comienzan por el primer campo y avanzan hacia la derecha sin saltar posiciones**.

Piensa en una agenda ordenada por «país, ciudad, apellido». Encontrar todas las personas de España resulta natural. Encontrar las de España, Madrid también. Encontrar solamente todas las personas apellidadas García, sin conocer país ni ciudad, ya no aprovecha esa misma organización de la misma forma.

Por tanto, un índice que empieza por `estado` es una buena opción para consultas que filtran por estado, pero **no debemos asumir que es igual de eficiente** al consultar exclusivamente por importe.

Hay matices: MongoDB puede escoger otros planes o incluso recorrer un índice completo en algunos casos. La regla del prefijo no significa «prohibido usarlo» sino «la estructura ordenada no proporciona el mismo acceso selectivo si omites sus primeros campos».

4. E, S y R: tres exigencias distintas

La pauta conocida como **ESR** ayuda a decidir el orden de campos en muchos índices compuestos. No es una fórmula mágica. Son iniciales de tres conceptos:

Letra	Significado	En nuestro ejemplo
E	*Equality* (igualdad)	`estado: "pagada"`
S	*Sort* (ordenación)	`sort({ fecha: -1 })`
R	*Range* (rango)	`importe: { $gt: 500 }`

Con ese criterio, un índice candidato es:

```
db.ventas.createIndex(
  { estado: 1, fecha: -1, importe: 1 },
  { name: "ix_esr" }
)
```

Léelo así: organizamos primero por el estado exacto; dentro de las pagadas podemos recorrer fechas de más reciente a más antigua; y además incorporamos el importe como tercera parte de la clave.

Cuidado importante: colocar `importe` después de `fecha` no significa que MongoDB pueda saltar de golpe a todas las ventas superiores a 500 sin inspeccionar otras claves del mismo grupo. Al priorizar la ordenación por fecha, el rango final puede requerir examinar más entradas del índice y filtrar importes. Esa es la contrapartida que hay que medir.

5. ¿Y si el rango es más importante que la ordenación?

Podríamos crear el índice en otro orden:

```
db.ventas.createIndex(
  { estado: 1, importe: 1, fecha: -1 },
  { name: "ix_ers_practica" }
)
```

Ahora primero agrupamos por estado, y dentro de cada estado ordenamos por importe. Es más fácil localizar directamente importes superiores a un umbral. Pero las fechas quedan organizadas **dentro de cada valor de importe**, no necesariamente de forma global entre todas las ventas que cumplen el rango.

Eso puede obligar al motor a ordenar los resultados con una operación adicional, que en `explain` puede aparecer como una etapa `SORT`. Ordenar no es necesariamente malo, pero consume recursos y, en consultas grandes, puede volverse caro.

Esta variante a veces se llama **ERS**, como contraste con ESR. **No elijas una por su nombre**: piensa si tu consulta necesita recuperar un conjunto pequeño mediante el rango o entregar pocos resultados ya ordenados. El optimizador puede escoger distintos planes según datos, índices disponibles y estadísticas.

6. ¿Qué indica realmente una etapa SORT?

Ordenar los documentos es una operación lógica. Un índice adecuado puede suministrarlos en el orden solicitado, y entonces el plan no necesita una etapa de ordenación independiente. En otros casos, MongoDB debe reunir candidatos y ordenarlos antes de entregar los primeros diez.

Una etapa `SORT` no demuestra por sí sola un problema grave. Debemos preguntarnos:

- ¿Cuántos candidatos hay que ordenar?
- ¿Cuántas claves y documentos se examinan?
- ¿Hay que recuperar documentos con `FETCH`?
- ¿De verdad devuelve las diez ventas solicitadas, en orden correcto?

No compares solo milisegundos. En un ordenador con caché caliente, una ordenación adicional puede parecer barata; con más volumen o carga, la situación puede cambiar.

7. Qué significa comparar ESR y ERS de manera limpia

Para investigar el orden de índices, mantenemos:

1. La **misma colección** y el mismo número de documentos.
2. El **mismo filtro** por estado e importe.
3. La **misma ordenación** por fecha.
4. El **mismo límite** de diez resultados.

Solo variamos el índice utilizado. Para controlar la comparación podemos emplear `hint("ix_esr")` y `hint("ix_ers_practica")`. `hint` le pide al motor que pruebe un índice concreto; aquí sirve como instrumento de aprendizaje, **no como recomendación de forzar siempre planes en producción**.

En las prácticas se prepararán ambos índices, se repetirá la consulta y se comprobará que los resultados son los mismos, aunque el plan y las cantidades examinadas puedan ser diferentes.

8. Preguntas para pensar antes de la práctica

Pregunta A. Dispones de `{ estado: 1, fecha: -1 }`. ¿Por qué resulta natural filtrar por `estado` y ordenar por `fecha`, pero no necesariamente buscar solo por `fecha`?

Pregunta B. ¿Por qué un índice `{ estado: 1, importe: 1, fecha: -1 }` puede ayudar con un filtro de importes, pero aun así necesitar una ordenación adicional por fecha?

Pregunta C. Si ambos índices devuelven las mismas diez ventas, ¿cómo decidirías cuál hizo menos trabajo?

Pregunta D. ¿Sería justo comparar ESR con diez resultados y ERS con cien? Explica tu razonamiento.

Respuestas orientativas

- **A:** el primer índice está agrupado por estado, así que sin fijar el primer campo la organización por fecha no equivale a una ordenación global simple.
- **B:** al recorrer múltiples importes, las fechas se encuentran dentro de bloques con valores diferentes de importe; no forman necesariamente una única secuencia descendente.
- **C:** observaría claves/documentos examinados, presencia de ordenación y las necesidades de la aplicación, además del tiempo medido en varias ejecuciones.
- **D:** no; habría cambiado también el trabajo pedido. La comparación debe mantener el mismo filtro, ordenación y límite.

Puente hacia la práctica

No es necesario memorizar «ESR siempre es mejor». Debes poder defender la elección: **qué campo filtra por igualdad, qué campo ordena, cuál representa un rango, y qué sacrificio aceptas**. El laboratorio siguiente hará visibles estas decisiones.

Fuente para ampliar: https://www.mongodb.com/docs/v8.0/tutorial/equality-sort-range-rule/

Unidad 3. Cardinalidad, selectividad y coste: por qué algunos índices ayudan poco

3.1. Un problema distinto del número de venta

El índice por número de venta parecía una elección obvia: un número identifica una sola venta. Ahora el equipo comercial pregunta «¿cuántas ventas están pagadas?». Podríamos pensar que cualquier campo usado en un filtro merece un índice. Pero `estado` solo toma cuatro valores en nuestra colección sintética: `pagada`, `pendiente`, `enviada` y `cancelada`. Cada uno aparece muchas veces.

Si existen 200.000 documentos distribuidos entre cuatro estados, filtrar uno de ellos podría seleccionar en torno a 50.000 documentos. No es lo mismo que buscar un número que identifica uno solo.

Llamamos **cardinalidad** al número de valores distintos que aparece en un campo. El campo `numero` tiene cardinalidad muy alta: prácticamente cada documento contiene un valor diferente. `estado` tiene cardinalidad baja: se repiten pocos valores. La cardinalidad, sin embargo, **no basta por sí sola para juzgar un índice**. También importa cuántos documentos devuelve una consulta concreta, su ordenación, el límite y si necesita leer otros campos.

3.2. Selectividad: qué fracción sobrevive al filtro

Piensa en una criba. Si lanzamos 200.000 ventas y el filtro `numero:199998` deja pasar una, selecciona una fracción diminuta. Si el filtro `estado:"pagada"` deja pasar 50.000, mantiene una cuarta parte.

Al hablar de **selectividad**, nos referimos a lo restrictivo que resulta el filtro. No confundas los conceptos:

- **Cardinalidad** caracteriza los valores posibles del campo en los datos.
- **Selectividad** caracteriza cuántos documentos deja pasar una condición de búsqueda determinada.

Un campo con pocos valores puede contener uno extraordinariamente raro. Por ejemplo, `estado:"en_revision_legal"` podría existir solo en diez ventas aunque `estado` tenga unos pocos estados en total. Ese valor concreto sí sería muy selectivo.

MongoDB puede decidir que para una consulta de baja selectividad resulta poco ventajoso utilizar cierto índice, especialmente si después debe recuperar gran cantidad de documentos. El optimizador elige un plan entre los disponibles; no está obligado a usar el índice que a nosotros nos parezca más bonito.

3.3. ¿Cómo medimos el problema?

Ejecutamos la misma consulta y observamos `explain("executionStats")`:

```
db.ventas.find({ estado: "pagada" }).explain("executionStats")
```

Lo primero es comprobar `nReturned`. Si devuelve aproximadamente una cuarta parte de toda la colección, el usuario está pidiendo un conjunto grande. Aun con un índice por `estado`, MongoDB deberá hacer trabajo para producir todos esos documentos.

Ahora añadimos una segunda condición:

```
db.ventas.find({
  estado: "pagada",
  numero: 199998
}).explain("executionStats")
```

Aquí el número reduce el conjunto de candidatos a un máximo de uno. La combinación de filtros y los índices disponibles importan más que el nombre del campo aislado.

Pregunta diagnóstica: si quieres devolver 50.000 documentos completos, ¿sería realista esperar `totalDocsExamined=1`? Evidentemente no. Un índice no cambia el hecho de que el cliente ha pedido muchos documentos.

3.4. Leer resultados y leer candidatos

Es fácil confundir `nReturned` con `totalDocsExamined`. Si una consulta devuelve 10.000 ventas y examina 10.000 documentos, la relación entre ambos números puede ser razonable. Si devuelve una y examina 200.000, merece investigación.

No interpretes esos contadores con una regla mecánica de «cuanto menor, mejor». Considera también `totalKeysExamined`, el coste de ordenación y el volumen de información retornada.

Para explicar una consulta a otra persona usa este orden:

1. **Qué necesitaba la aplicación.**
2. **Qué devolvió MongoDB y si era correcto.**
3. **Qué camino utilizó el motor.**
4. **Cuántos candidatos examinó.**
5. **Qué alternativa medirías y qué costes introduciría.**

3.5. El precio invisible de escribir

Imaginemos que la empresa recibe 500 ventas cada segundo. Cada inserción actualiza la colección y los índices que correspondan. Si añadimos decenas de índices por campos apenas consultados, aumentamos el trabajo de escritura, el espacio utilizado y la planificación de mantenimiento.

No hay una respuesta universal como «tres índices por colección». El presupuesto de índices depende del patrón de uso, tamaño de documentos, frecuencia de escritura, memoria disponible y consultas críticas. El buen diseño busca un equilibrio.

Ejercicio razonado: una aplicación consulta por `numero` miles de veces al día, por `clienteId` cientos de veces y por `colorDeEtiqueta` una vez al trimestre. ¿Cuál estudiarías primero? La respuesta se apoya en las consultas reales: el índice por número probablemente es prioritario; para el campo raramente consultado primero medirías si merece la pena mantenerlo.

Unidad 4. Índices multikey: buscar dentro de listas

4.1. Cuando un campo deja de contener un único valor

Hasta ahora teníamos campos como `estado`, que contiene una cadena, o `importe`, que contiene un número. En la práctica también existen **arrays**: listas de valores dentro de un documento. Una venta puede tener etiquetas como estas:

```
{
  numero: 348,
  etiquetas: ["venta", "web", "urgente"]
}
```

La empresa pregunta: «¿Qué ventas contienen la etiqueta `urgente`?». No está preguntando por una cadena completa que represente el array, sino por **la presencia de un elemento dentro de una lista**.

MongoDB permite expresarlo de una forma directa:

```
db.ventas.find({ etiquetas: "urgente" })
```

En el conjunto de datos preparado, unas ventas incluyen esa etiqueta y otras no. Como el campo contiene un array, MongoDB puede crear automáticamente un **índice multikey** cuando indexamos dicho campo:

```
db.ventas.createIndex({ etiquetas: 1 })
```

No existe una palabra `multikey` dentro del comando: MongoDB reconoce que está indexando valores de arrays.

4.2. ¿Qué significa «varias claves»?

Imagina una ficha comercial que contiene tres etiquetas: `venta`, `web` y `urgente`. Para facilitar la búsqueda de cualquier etiqueta, el índice puede representar distintas entradas derivadas de los elementos del array. El documento no se duplica en la colección; lo que crece son las entradas asociadas al índice.

Eso explica dos hechos que a menudo sorprenden:

- El número de **claves** examinadas puede ser superior al número de **documentos** devueltos.
- Cambiar el contenido de una lista indexada puede implicar actualizar varias entradas.

No interpretes automáticamente `totalKeysExamined > nReturned` como un error. En un índice multikey es normal que los arrays participen en varias claves.

4.3. Dos condiciones en un array: una trampa frecuente

Si un array almacena documentos, por ejemplo:

```
productos: [
  { sku: "SKU10", cantidad: 1 },
  { sku: "SKU20", cantidad: 3 }
]
```

podríamos querer encontrar ventas que contengan **un mismo producto** con SKU10 y cantidad 3. Un filtro ingenuo sobre `productos.sku` y `productos.cantidad` puede satisfacer cada condición en elementos diferentes del array. Esto significa que podríamos aceptar la venta aunque SKU10 tenga cantidad 1 y quien tenga cantidad 3 sea SKU20.

Para indicar que las dos condiciones deben cumplirse dentro **del mismo elemento**, MongoDB ofrece `$elemMatch`:

```
db.ventas.find({
  productos: {
    $elemMatch: { sku: "SKU10", cantidad: 3 }
  }
})
```

Antes de crear índices, debemos comprender la semántica de la consulta. **Un índice no puede corregir una pregunta formulada de manera equivocada.**

4.4. Limitaciones de los compuestos multikey

MongoDB impone restricciones cuando un índice compuesto incluiría varios campos que son arrays en un mismo documento. La razón es estructural: combinar independientemente listas podría generar demasiadas combinaciones de claves y ambigüedades en la indexación.

No conviertas esto en «los índices multikey no se pueden combinar». Sí se pueden crear índices compuestos que involucren un campo array y otros campos escalares cuando se respetan las restricciones. Lo importante es revisar el esquema real antes de crear uno.

4.5. Cómo observar un multikey

En la salida de `explain` de un plan con índice, busca propiedades como `isMultiKey` en los lugares pertinentes del plan, junto a las etapas `IXSCAN`. No dependas de que todas las versiones impriman exactamente el mismo árbol de campos: los detalles del plan pueden cambiar.

Ejercicio de interpretación: si `etiquetas:"urgente"` encuentra 18.000 ventas, ¿es seguro que el índice hará que MongoDB examine solo una clave? No. Podría necesitar recorrer muchas entradas de índice porque `urgente` aparece en muchas ventas.

Comprobación: diferencia entre una lista de valores, una lista de subdocumentos y un campo escalar. Explica para qué sirve `$elemMatch` sin usar la expresión «hace más rápido».

Unidad 5. Búsqueda de texto: encontrar palabras, no números exactos

5.1. Una consulta que no es igualdad

Una venta puede contener una descripción como `cafe premium en grano`. Si consultas `descripcion:"cafe"`, MongoDB busca un valor completo igual a la cadena `cafe`, no necesariamente un documento cuya descripción contiene esa palabra.

La búsqueda textual plantea un problema distinto: el cliente quiere documentos relacionados con **términos**, no solo cadenas idénticas. Para mostrarlo en el laboratorio tenemos descripciones como `cafe premium en grano`, `bicicleta urbana plegable` o `reclamacion urgente del cliente`.

Un índice textual clásico se define así:

```
db.ventas.createIndex(
  { descripcion: "text" },
  { name: "ix_descripcion_texto", default_language: "spanish" }
)
```

La clave `"text"` indica el tipo de índice. La opción de idioma influye en el procesamiento lingüístico. Eso significa que no debemos confundir búsqueda textual con una simple búsqueda de subcadenas de caracteres.

5.2. Consultar términos

```
db.ventas.find({
  $text: { $search: "cafe" }
})
```

El operador `$text` pide búsqueda textual sobre la colección, cuando existe un índice adecuado. No hay que esperar que funcione como una comparación exacta de texto completo.

El sistema puede procesar palabras considerando reglas lingüísticas, como formas y términos frecuentes. Para comprender por qué un resultado aparece y otro no, revisa el idioma y los términos almacenados. Nunca expliques que «busca cualquier secuencia de letras en cualquier lugar»: esa es una descripción equivocada.

5.3. Relevancia y textScore

Si muchos documentos contienen términos relacionados, puede interesarnos cuál es más relevante para la búsqueda. MongoDB permite proyectar la puntuación textual:

```
db.ventas.find(
  { $text: { $search: "cafe" } },
  { descripcion: 1, score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

La puntuación ayuda a ordenar coincidencias según el mecanismo textual disponible. **No es una probabilidad de verdad** ni indica la intención real del cliente: representa un cálculo de relevancia para esa consulta.

Búsqueda textual clásica frente a sistemas de búsqueda más amplios

Los índices `text` de MongoDB Community permiten determinados escenarios de búsqueda de términos. No debes atribuirles automáticamente las capacidades de un motor especializado moderno: analizadores complejos, tolerancia avanzada a errores, búsqueda semántica o ciertas funciones de MongoDB Search son temas diferentes y dependen del producto y despliegue.

5.4. Costes y decisiones

Un índice textual consume almacenamiento y añade trabajo de actualización cuando cambian descripciones. Puede ser útil para un buscador sencillo dentro de una colección. Pero si la aplicación necesita un motor de búsqueda muy sofisticado, tendremos que estudiar otras opciones. El ejercicio introductorio pretende comprender la diferencia entre igualdad textual, términos indexados y relevancia, no evaluar todos los motores de búsqueda del mercado.

Preguntas: ¿por qué no basta `descripcion:"cafe"` para encontrar `cafe premium`? ¿Qué información aporta `textScore` y qué no puede demostrar?

Unidad 6. Índices geoespaciales: buscar por cercanía

6.1. Del catálogo al mapa

Ahora imaginemos una aplicación que muestra puntos de entrega cerca del usuario. Un documento puede almacenar coordenadas geográficas. El reto no consiste en ordenar palabras, sino en responder «¿qué lugares están a menos de cierta distancia de este punto?».

Para datos GeoJSON, un punto se representa así:

```
{
  nombre: "Punto A",
  ubicacion: {
    type: "Point",
    coordinates: [-3.7038, 40.4168]
  }
}
```

Orden decisivo: las coordenadas son [longitud, latitud]. Invertirlas puede colocar el punto en una región completamente diferente. La longitud es el eje este-oeste y la latitud el eje norte-sur.

6.2. Qué aporta 2dsphere

Para trabajar con geometrías GeoJSON sobre una esfera terrestre aproximada, MongoDB ofrece índices `2dsphere`:

```
db.puntos.createIndex({ ubicacion: "2dsphere" })
```

Ese índice ayuda a resolver determinadas consultas espaciales, como encontrar puntos cercanos a una coordenada.

```
db.puntos.find({
  ubicacion: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [-3.7038, 40.4168]
      },
      $maxDistance: 3500
    }
  }
})
```

El valor `3500` de `$maxDistance` se expresa en metros en esta consulta GeoJSON: solicitamos ubicaciones próximas dentro de aproximadamente 3,5 kilómetros. La cercanía sobre la superficie no equivale a tiempo de viaje por calles y carreteras. Un puente, un río o una montaña pueden alterar el trayecto real.

6.3. Qué validar antes de culpar al índice

Si obtienes resultados extraños, primero revisa:

- El orden longitud/latitud.
- El tipo GeoJSON `Point` y la estructura `coordinates`.
- La unidad de la distancia.
- La existencia del índice `2dsphere`.
- Si los puntos ficticios representan coordenadas realistas para el caso estudiado.

No confundas índices `2d` para coordenadas planas con los `2dsphere` para ubicaciones geográficas. El tipo de dato y la consulta deben corresponderse con el modelo espacial elegido.

6.4. Decisiones de diseño

Una empresa de mensajería podría indexar la ubicación de sus centros de distribución para buscar el más cercano a un cliente. Eso no resuelve automáticamente la ruta óptima de reparto. El índice contesta consultas espaciales sobre los datos almacenados; optimizar recorridos con múltiples paradas y restricciones es otro problema.

Pregunta de transferencia: ¿qué fallaría si una aplicación intercambia latitud y longitud? ¿Por qué un punto situado a 2 kilómetros en línea recta no garantiza un trayecto de 2 kilómetros?

Unidad 7. Índices hashed: igualdad y distribución de valores

7.1. ¿Por qué una representación hash?

Un índice tradicional sobre un número lo organiza según su valor. Si buscamos entre 500 y 900, la ordenación numérica facilita localizar ese intervalo. Pero hay sistemas donde el requisito dominante no es recorrer valores cercanos, sino encontrar por igualdad y ayudar a distribuir las claves de forma más uniforme.

Un índice `hashed` almacena una representación hash del valor indexado. En términos intuitivos, una función hash transforma un valor en otro valor calculado de forma determinista. Si aplicamos la función al mismo dato, obtenemos el mismo resultado; valores próximos en el espacio original no tienen por qué quedar próximos en el espacio transformado.

En el laboratorio existe el campo `idGrupo`, que identifica uno entre muchos grupos sintéticos:

```
db.ventas.createIndex({ idGrupo: "hashed" })
```

La consulta típica es una igualdad:

```
db.ventas.find({ idGrupo: 809 })
```

El sistema calcula el hash del valor buscado y puede utilizar el índice para encontrar candidatos adecuados.

7.2. Qué no debes pedirle a un hash

La idea de hash rompe la correspondencia con el orden original. Los valores `500`, `501` y `502` no tienen por qué aparecer juntos en el índice hashed. Por ello, **no es una estructura diseñada para acelerar rangos numéricos** como «grupos entre 500 y 900».

```
db.ventas.find({ idGrupo: { $gte: 500, $lte: 900 } })
```

Este filtro puede ser perfectamente válido como consulta, pero no debes suponer que el índice hashed es el acceso adecuado para resolver su intervalo. La propiedad que estamos aprovechando es igualdad, no ordenación.

7.3. No confundir hash de índice con cifrado de contraseñas

Que un índice utilice la palabra *hashed* no significa que los datos de la base estén cifrados, ni que podamos almacenar contraseñas en claro y confiar en el índice. Son problemas completamente distintos.

Una función hash usada para organizar claves no es un mecanismo de protección de secretos. El propósito aquí es la representación de valores para determinadas operaciones de acceso y distribución.

Pregunta: ¿utilizarías `{ idGrupo:"hashed" }` para mostrar los grupos ordenados de menor a mayor? Explica por qué un índice ordenado convencional encaja mejor con esa tarea.

Unidad 8. Índices únicos y parciales: la integridad también se diseña

8.1. Una pregunta distinta a la rapidez

Supón que cada empleado debe tener una dirección de correo distinta. Un índice que permita localizar cuentas por correo será útil para acceder rápidamente. Pero la empresa exige algo más: **no permitir que se creen dos cuentas con el mismo correo**.

Eso es una regla de integridad. MongoDB puede imponerla con un índice único:

```
db.alumnos.createIndex({ email: 1 }, { unique: true })
```

Si insertas dos documentos con `email:"ana@ejemplo.test"`, el segundo intento debe fallar por clave duplicada, normalmente con el código `E11000`.

El fallo es el resultado esperado. No hay que interpretar que MongoDB «no sabe insertar». Está rechazando datos que violan una regla declarada por nosotros.

8.2. Crear una restricción sobre datos que ya existen

¿Qué ocurre si la colección ya contiene correos repetidos y después intentamos crear un índice único? MongoDB no puede imponer limpiamente esa restricción sin resolver antes los duplicados existentes: la construcción del índice puede fallar.

El orden correcto de implantación en una base real incluye:

1. Identificar qué campo se considera único y bajo qué reglas.
2. Buscar documentos duplicados o valores ausentes.
3. Resolver esos conflictos de acuerdo con el negocio.
4. Crear la restricción.
5. Probar tanto una inserción válida como una duplicada.

No borres datos reales para conseguir que pase el comando. Las reglas de unicidad son decisiones de negocio, no simples ajustes de rendimiento.

8.3. ¿Qué ocurre con valores que faltan?

Una complicación importante aparece si algunas cuentas aún no tienen correo electrónico. Un índice único de un solo campo trata de forma especial los documentos que no contienen ese campo o contienen `null`. Puede impedir crear múltiples documentos sin correo bajo determinadas configuraciones.

Por eso no deberíamos decidir «índice único en email» sin preguntarnos si el campo es obligatorio para todos, opcional para algunos o si solo debe ser único cuando realmente contiene un valor.

8.4. Índices parciales: solo ciertos documentos

Una empresa quiere una clave única para los **cupones activos**, pero quizá permite reutilizar códigos de promociones históricas inactivas. Podemos estudiar un índice parcial que solo se aplique a documentos que cumplan un filtro:

```
db.cupones.createIndex(
  { codigo: 1 },
  {
    unique: true,
    partialFilterExpression: { activo: true }
  }
)
```

La intención es exigir unicidad de `codigo` entre los documentos donde `activo` sea verdadero. Al diseñarlo debemos pensar qué significa un documento sin `activo` y qué ocurrirá cuando un cupón pase de inactivo a activo.

Por ejemplo:

- Cupón A, código `PROMO7`, activo: verdadero.
- Cupón B, código `PROMO7`, activo: falso.
- Intento de activar B mientras A sigue activo.

Ese último cambio debe analizarse como una posible violación de la restricción. El índice no solo interviene al insertar: también importa cuando actualizamos un documento de manera que entra en el conjunto indexado.

8.5. Índice parcial no es índice sparse

Ambos pueden almacenar únicamente una parte de los documentos, pero **no son equivalentes**. Un índice parcial utiliza una expresión declarada para delimitar qué documentos se incluyen; uno sparse tiene reglas específicas relacionadas con la presencia de campos. No uses las palabras como sinónimos al documentar una arquitectura.

Pregunta razonada: si las promociones deben ser únicas solo mientras están activas, ¿por qué un índice único general sobre `codigo` podría ser demasiado restrictivo? ¿Qué riesgo aparece cuando se reactiva un cupón?

Unidad 9. TTL: datos que deben caducar solos

9.1. El problema de las sesiones olvidadas

Imagina una aplicación que almacena documentos temporales para una sesión de usuario. Pasados ciertos minutos, esos datos ya no deberían conservarse. Podríamos programar una tarea externa que buscara registros antiguos y los borrara. MongoDB ofrece otra posibilidad: un índice **TTL**, de *time to live*, asociado a un campo de fecha.

En una colección `sesiones` cada documento podría contener:

```
{
  _id: "sesion-7",
  usuario: "alumno-1",
  expiraEn: ISODate("2025-11-01T10:00:00Z")
}
```

Si `expiraEn` representa el instante absoluto en que deja de ser válida, creamos:

```
db.sesiones.createIndex(
  { expiraEn: 1 },
  { expireAfterSeconds: 0 }
)
```

El valor cero indica que, una vez alcanzado el instante almacenado en el campo, el documento resulta elegible para la eliminación TTL.

9.2. Caducar no equivale a desaparecer en ese milisegundo

MongoDB ejecuta la eliminación TTL mediante un monitor periódico y asíncrono. **Un documento caducado puede permanecer un tiempo después de su fecha de expiración.** No construyas un sistema de autorización suponiendo que «si el documento existe es porque su sesión sigue vigente».

La aplicación debe comprobar la validez temporal cuando toma decisiones sensibles. El TTL ayuda a limpiar datos, no sustituye toda la lógica de seguridad y expiración.

En nuestro laboratorio, el parámetro de monitor TTL se acelera para hacer razonable la comprobación. Esa elección es específica de la demostración; no presupone la misma frecuencia en todos los servidores.

9.3. ¿Por qué es crucial usar una fecha?

Un campo como `"2025-11-01"` almacenado literalmente como cadena no representa el mismo tipo que una fecha BSON. Los índices TTL necesitan valores temporales adecuados para sus reglas de expiración. Si el documento usa un campo ausente o un tipo que no se considera fecha válida para TTL, no podemos esperar que el monitor lo trate igual.

Para comprobarlo en el laboratorio:

1. Insertamos un documento con una fecha ya vencida.
2. Insertamos otro con una fecha futura.
3. Observamos si el monitor elimina el primero.
4. Confirmamos que el segundo permanece.

La prueba debe contemplar un margen de espera. **No concluyas que TTL está roto porque el documento no desaparezca al instante.**

9.4. Un error de modelado: confundir creación y expiración

Hay dos formas habituales de pensar la caducidad:

- Guardar **cuándo fue creado** y definir cuántos segundos debe durar.
- Guardar **cuándo debe expirar** e indexarlo con `expireAfterSeconds:0`.

No mezcles ambas ideas sin analizarlo. Si almacenas la fecha de creación y usas cero, estarás declarando que el documento está caducado desde que nace. Si almacenas una fecha futura y le añades un intervalo no deseado, lo borrarás más tarde de lo previsto.

Ejercicio conceptual: una sesión creada a las 10:00 debe expirar a las 10:30. Diseña el campo temporal y el índice de las dos estrategias. Después explica por qué el monitor no garantiza eliminarla exactamente a las 10:30:00.

Unidad 10. Consultas cubiertas, proyecciones y FETCH

10.1. Hasta ahora recuperábamos documentos completos

A veces una aplicación no necesita un documento entero. Supón que muestra una lista de clientes y sus importes, sin detalles de entrega ni descripciones largas. Si el índice contiene todas las claves solicitadas y el filtro puede resolverse desde él, MongoDB podría obtener los resultados **sin recuperar los documentos completos**.

A eso se le llama consulta **cubierta** por el índice, bajo las condiciones correspondientes.

Imagina un índice:

```
db.ventas.createIndex({ clienteId: 1, importe: 1 })
```

La consulta solo pide esos dos campos:

```
db.ventas.find(
  { clienteId: 809 },
  { _id: 0, clienteId: 1, importe: 1 }
)
```

La proyección es el segundo objeto que pasamos a `find`. `_id:0` excluye expresamente el identificador `_id`, que MongoDB incluye por defecto en muchos resultados. Si no lo excluyéramos y el índice no contuviera `_id`, la cobertura completa de esta consulta podría verse impedida.

10.2. Qué observar en explain

Si una consulta está realmente cubierta, los contadores pueden mostrar `totalDocsExamined:0`: el motor obtiene los datos de las entradas de índice sin recuperar documentos de la colección.

Ese cero es **muy significativo**, pero no significa que el trabajo sea gratuito. El motor sigue recorriendo claves, procesando filtros y devolviendo resultados. Para entenderlo también miramos `totalKeysExamined`.

Añadir un campo a la proyección cambia las necesidades. Si de pronto pedimos `descripcion` y el índice solo contiene cliente e importe, probablemente será necesario recuperar los documentos para conocer la descripción.

10.3. Cubrir todas las consultas puede ser mala idea

Podríamos sentir la tentación de añadir muchos campos a los índices para que más consultas queden cubiertas. Eso aumenta el tamaño de los índices y el coste de mantenerlos. Además, una consulta con arrays y multikey puede tener reglas adicionales que impidan ciertas coberturas.

El diseño correcto considera la frecuencia de uso de la consulta, su proyección real, la cantidad de resultados y el coste de escritura. No se trata de ganar un concurso de índices enormes.

Pregunta: ¿por qué una consulta puede examinar cero documentos y, sin embargo, examinar 500 claves? ¿Qué cambiaría al pedir el campo `descripcion`?

Unidad 11. Mantener índices y decidir cuáles merece la pena conservar

11.1. Saber qué índices existen

Antes de proponer uno nuevo, comprueba el inventario de la colección:

```
db.ventas.getIndexes()
```

Esta orden muestra definiciones de índices, nombres y propiedades. No te dice por sí sola si cada índice resulta valioso ni cuánto acelera cada consulta. Esa conclusión exige observar cargas y consultas reales.

Puedes encontrar, por ejemplo, un índice compuesto cuyo prefijo es muy parecido a otro índice simple. En determinadas condiciones el compuesto puede atender consultas que antes usaban el simple, pero antes de eliminarlo hay que analizar diferencias como unicidad, opciones y cargas específicas.

11.2. Índices ocultos: ensayar una retirada

En algunas versiones y configuraciones podemos **ocultar** un índice para impedir que el optimizador lo elija en planes normales, sin eliminar su definición. Esto permite estudiar cómo se comportan las consultas si dejamos de ofrecerlo al planificador:

```
db.ventas.hideIndex("ix_cliente")
```

Después podríamos volver a mostrarlo:

```
db.ventas.unhideIndex("ix_cliente")
```

Ocultar no significa liberar su espacio o dejar de mantenerlo. El índice sigue existiendo, y el servidor debe conservar su consistencia. No confundas «no disponible para el planificador» con «eliminado de la colección».

11.3. Eliminar un índice es una decisión con riesgo

```
db.ventas.dropIndex("ix_cliente")
```

Eliminar un índice puede reducir mantenimiento y espacio, pero también puede empeorar consultas que dependían de él. En un sistema real conviene conocer métricas de uso, necesidades del producto y ventanas de mantenimiento. No se recomienda borrar índices en producción solo para comprobar una idea de clase.

En las prácticas usamos exclusivamente colecciones de demostración que podemos reconstruir.

11.4. Qué queremos aprender al medir un millón de ventas

Si repetimos el mismo experimento con 200.000 y 1.000.000 de documentos, no estamos buscando un número mágico de milisegundos. Queremos razonar sobre cómo crece el trabajo de un recorrido completo y cómo un índice muy selectivo evita recorrer todo el conjunto.

Para que la comparación tenga sentido debemos documentar:

- mismo patrón de consulta;
- cantidad exacta de datos;
- índice disponible o ausente;
- resultados devueltos;
- claves y documentos examinados;
- entorno de ejecución;
- diferencias de planificación o caché que puedan explicar tiempos.

Conclusión del capítulo: un índice se justifica por las necesidades de consulta y la evidencia de su efecto, no por su mera existencia. La persona que administra la base debe saber explicar qué problema resuelve y qué coste agrega.

Comprobación integrada

Una aplicación tiene tres operaciones: buscar un pedido por número, mostrar los últimos pedidos pagados y localizar etiquetas de riesgo en una lista. Propón un índice candidato para cada una y explica la función del campo o del orden de campos. A continuación indica qué revisarías con `explain` y cómo estimarías el coste de mantener esos índices ante nuevas ventas.

No necesitas acertar todos los detalles de un plan sin ejecutarlo. Necesitas formular una hipótesis comprobable.

Unidad 12. Qué ocurre dentro de un índice: del libro al árbol B

12.1. Por qué no basta con guardar una lista ordenada en cualquier sitio

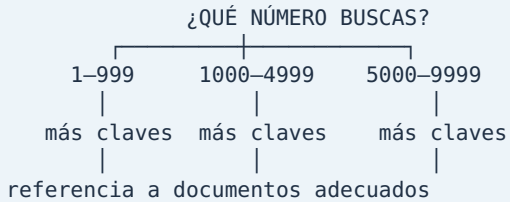
Hasta ahora hemos comparado un índice con un catálogo alfabético. Esa imagen es útil, pero deja una pregunta sin contestar: si hay un millón de claves, ¿cómo puede el motor localizar rápidamente una zona pequeña sin leerlas todas desde el comienzo?

Una manera ingenua sería guardar todos los números de venta en una lista y recorrerla hasta encontrar el número deseado. Pero eso sería otro recorrido secuencial, ahora sobre una lista en lugar de una colección. Para evitarlo se necesita una organización que permita **descartar grandes regiones de claves** en cada decisión.

Los índices ordenados de MongoDB se basan en estructuras de tipo árbol B, que mantienen las claves organizadas en una forma adecuada para búsquedas y recorridos eficientes. No necesitamos implementar un árbol desde cero, pero sí comprender la intuición.

12.2. El árbol como mapa de intervalos

Imagina que la primera página de un índice permite decidir entre tres intervalos:



Este dibujo **no reproduce la estructura física exacta de un índice MongoDB**. Representa una idea: cada decisión reduce el espacio de búsqueda. En un árbol B real, los nodos contienen varias claves y referencias que guían el descenso y facilitan recorridos ordenados.

Si buscas la venta 8.700, no necesitas examinar todas las entradas correspondientes a 1–999. Puedes avanzar por la parte del árbol correspondiente al intervalo que incluye 8.700.

12.3. Igualdad y rango recorren el árbol de forma diferente

Una igualdad como:

```
db.ventas.find({numero:8700})
```

puede dirigirse a la región donde estaría la clave buscada.

En cambio, un rango pide varias claves consecutivas:

```
db.ventas.find({
  numero:{$gte:8700,$lt:8750}
})
```

El motor localiza el inicio del intervalo y puede recorrer las entradas correspondientes hasta el final. Por eso un índice ordenado es útil no solo para identificar una venta, sino también para consultas por rangos.

No concluyas que toda consulta con `$gte` es eficiente: si el intervalo cubre casi la colección entera, puede haber que recorrer muchísimas claves. La selectividad sigue importando.

12.4. De una clave al documento: por qué aparece FETCH

La estructura de índice mantiene claves y referencias que permiten localizar documentos. Si la aplicación pide campos que no están en el índice, MongoDB necesita recuperar esos documentos.

Por ejemplo, suponiendo que solo existe un índice por `numero`:

```
db.ventas.find({numero:8700},{numero:1,descripcion:1})
```

Aunque el índice localice la venta por número, la descripción debe obtenerse del documento. Esa recuperación explica conceptualmente una etapa `FETCH`. No se debe confundir `IXSCAN` con «la consulta nunca toca la colección».

12.5. Por qué las inserciones requieren trabajo

Si agregas una venta con número 8.701, la estructura debe incorporar esa nueva clave en su lugar correspondiente, manteniendo su organización. Con muchas inserciones, modificaciones y eliminaciones, el índice exige mantenimiento.

El coste exacto depende de detalles de almacenamiento, carga de trabajo y estructura de datos. Para el alumno basta con comprender la causalidad: **el índice facilita ciertos accesos porque mantiene información adicional organizada; mantenerla no es gratis**.

12.6. ¿Qué ocurre cuando muchos documentos comparten una clave?

Imagina un índice ordenado por `estado`. Miles de ventas tienen el valor `pagada`. La organización del índice puede localizar la región de ese estado, pero si pedimos todas las pagadas sigue habiendo un conjunto enorme de coincidencias.

No existe magia: un índice no convierte miles de respuestas en una sola. La estructura ayuda a **localizar candidatos**, pero la aplicación debe pagar el coste de procesar y entregar lo que solicita.

Ejercicio razonado

Un alumno afirma que «como el índice es un árbol, cualquier consulta lee tres nodos y termina». ¿Qué error conceptual contiene? Respuesta: el número de pasos depende de la estructura y del número de claves que cumplan la consulta; además puede haber recuperación de documentos, ordenación y procesamiento adicional. El árbol facilita el acceso, no elimina todas las operaciones.

Unidad 13. El optimizador de consultas y la lectura crítica de explain

13.1. Tener un índice no obliga a utilizarlo

Supón que la colección posee índices por `numero`, `estado` y `fecha`. Para una búsqueda concreta, MongoDB evalúa planes posibles y selecciona uno según la información y los mecanismos disponibles.

No debemos imaginar que la aplicación escribe «utiliza ix_numero» cada vez que busca una venta. Normalmente escribimos una **consulta declarativa**: especificamos qué queremos obtener. El servidor decide cómo obtenerlo.

Esa distinción explica por qué debemos comprobar los planes en lugar de asumir que el índice que acabamos de crear será siempre el elegido.

13.2. Cómo se representa un plan

Al llamar a:

```
db.ventas.find({numero:199998}).explain("executionStats")
```

la respuesta contiene información de planificación y estadísticas. Puede haber diferentes capas del plan; las etapas más útiles para empezar son:

- `COLLSCAN`: recorrido de documentos.
- `IXSCAN`: recorrido de entradas de índice.
- `FETCH`: recuperación de documentos a partir de candidatos.
- `SORT`: ordenación independiente cuando no se obtiene directamente del acceso elegido.
- `LIMIT` u otras etapas relacionadas con la cantidad devuelta.

No memorices una sola forma textual del JSON. El formato exacto de `explain` y la representación de sus etapas puede variar entre versiones y motores de ejecución. Busca el significado de las operaciones, no solo una palabra a una profundidad fija.

13.3. La diferencia entre plan ganador y estadísticas de ejecución

En una explicación encontramos datos sobre el plan elegido y, con `executionStats`, contadores derivados de la ejecución. La elección del plan responde a «¿qué camino cree adecuado el motor?». Los contadores responden a «¿qué trabajo observamos al ejecutarlo?».

Un `IXSCAN` puede ser un camino apropiado, pero aun así examinar decenas de miles de claves para resolver una consulta poco selectiva. Por eso debemos combinar la información cualitativa del plan con cifras como `totalKeysExamined` y `totalDocsExamined`.

13.4. El peligro de interpretar una sola cifra

Imagina dos planes:

Métrica	Plan A	Plan B
Documentos devueltos	10	10
Claves examinadas	25	18.000
Documentos examinados	10	10
Ordenación independiente	No	Sí

En este escenario el Plan A parece evitar más trabajo para la misma consulta. Aun así, la decisión real requiere examinar carga, coste de mantener índices y repetibilidad de las pruebas.

Un alumno podría elegir Plan B porque una ejecución aislada mostró menos milisegundos. Tal vez la caché, el orden de pruebas o la carga del ordenador influyeron. **No debemos convertir una anécdota temporal en una ley de optimización.**

13.5. ¿Para qué sirve hint y cuándo evitarlo?

`hint` permite solicitar un índice concreto en una consulta:

```
db.ventas.find({clienteId:809})
  .hint("ix_cliente")
  .explain("executionStats")
```

En clase es útil para comparar alternativas manteniendo el resto constante. En producción, forzar un índice sin necesidad puede impedir que el optimizador utilice un plan mejor cuando cambian los datos.

Una práctica responsable distingue:

- **Experimento controlado:** utiliza `hint` para estudiar un índice.
- **Consulta normal de aplicación:** deja al servidor elegir, salvo que exista una razón medida y documentada para intervenir.

13.6. Planificar una investigación

Cuando alguien informa «esta pantalla va lenta», no crees índices inmediatamente. Primero pregunta:

1. ¿Qué operación es lenta y cuál es su consulta real?
2. ¿Con qué frecuencia se ejecuta?
3. ¿Qué filtros, proyección, ordenación y límite utiliza?
4. ¿Cuántos documentos devuelve y cuántos examina?
5. ¿Qué índices existen y cuáles podrían servir?
6. ¿Qué alternativa podemos probar sin alterar datos reales?

7. ¿Qué coste de mantenimiento tendría la mejora?

Este protocolo evita optimizaciones basadas en intuición o en nombres de índices inventados.

Unidad 14. Medir 200.000 y un millón de ventas sin falsear la conclusión

14.1. Por qué usamos dos volúmenes de datos

Con veinte ventas, casi cualquier operación parecerá instantánea en un ordenador moderno. Eso hace difícil observar las diferencias entre planes de acceso. Cargar 200.000 documentos y repetir el experimento con un millón permite comprobar cómo escala el trabajo lógico.

No buscamos dramatizar tiempos ni medir la velocidad máxima de tu ordenador. Queremos observar cómo cambian las cantidades examinadas cuando crece la colección.

14.2. Una comparación cuyo resultado tiene significado

Elige una consulta por un número situado cerca del final de la colección:

```
db.ventas.find({numero:199998})
```

Con 200.000 ventas, ese número existe. Sin un índice secundario por `numero`, MongoDB puede examinar un número grande de documentos. Después creas el índice y repites exactamente la búsqueda.

Para un millón de ventas, debes **elegir un número que pertenezca al nuevo conjunto**, como 999.998. La condición lógica es la misma —buscar una venta concreta por número—, pero no puedes mantener sin pensar un identificador fuera de contexto.

14.3. Qué debe registrarse

Una tabla de laboratorio útil incluiría:

Elemento del experimento	200.000 ventas	1.000.000 ventas
Número de documentos cargados	Confirmado por conteo	Confirmado por conteo
Filtro utilizado	Número próximo al final	Número próximo al final
Índice disponible	Sí / no	Sí / no
Plan	COLLSCAN / IXSCAN	COLLSCAN / IXSCAN
Documentos examinados	Medición real	Medición real
Claves examinadas	Medición real	Medición real
Tiempo observado	Dato orientativo	Dato orientativo

El paso de 200.000 a un millón multiplica por cinco el volumen, pero eso no autoriza a afirmar de antemano que todos los tiempos se multiplicarán por cinco. El rendimiento real depende de recursos y de lo que el motor tiene que hacer.

14.4. Errores que invalidan el experimento

Una comparación deja de ser interpretable si:

- El conjunto «sin índice» aún conserva el índice de una práctica previa.
- Se busca un valor ausente y se compara con una búsqueda que sí devuelve documentos.
- Se cambia la proyección de una ejecución a otra.
- En una consulta se utiliza un límite y en la otra no.
- Se comparan ordenadores, discos o condiciones de carga diferentes sin registrarlo.
- Se promete una aceleración fija a partir de una sola ejecución temporal.

Una buena práctica puede mostrar que algo NO mejora, si el índice elegido no se adapta a la consulta o si el coste de obtener muchos resultados domina el trabajo. Eso también es aprendizaje.

14.5. La evidencia honesta

Para justificar una decisión de índice, presenta una afirmación verificable:

> «Para una búsqueda por número que devuelve un documento, la prueba con esta colección y este plan examinó muchos más documentos sin el índice que con el índice. Los tiempos observados son específicos de la ejecución».

No la sustituyas por una afirmación absoluta:

> «Un índice siempre hace que MongoDB sea cientos de veces más rápido».

La primera se apoya en datos; la segunda ignora el tipo de consulta, el volumen devuelto, la caché y el coste de mantener la estructura.

Caso final de razonamiento

Una tienda necesita consultar una venta exacta, listar ventas de un estado ordenadas por fecha, buscar texto en descripciones y encontrar puntos próximos. Explica por qué sería incorrecto resolver los cuatro problemas mediante un único índice `{numero:1}`. Propón el tipo de índice que estudiarías para cada uno y cómo comprobarías su utilidad antes de desplegarlo.